

# システム要件定義書

HitArrow風 Web/PWAカジュアルゲームアプリケーション開発計画

作成日: 2026年5月31日

対象ツール: Cursor (AIコードエディタ) 読込用仕様書

## 1. プロジェクト概要

本プロジェクトは、Google Play等で展開されている人気カジュアルゲーム「HitArrow（ヒットアロー）」の直感的かつ中毒性の高いゲームシステムをベースに、モダンなWeb技術を用いてリビルドする開発計画である。PCおよびスマートフォン（PWA化によるホーム画面起動）の双方でシームレスかつ高速に動作する環境を目指す。

### ゲームの基本コアメカニクス

- 回転する標的とタイミング:** 画面中央で回転する円形の標的（ターゲット）に対し、画面下部からタイミングよく「矢（アロー）」を射出して突き刺す。
- 衝突回避ルール:** すでに的に刺さっている他の矢に、新しく放った矢が接触（衝突）した時点でゲームオーバーとなる。
- ステージクリア条件:** 各ステージで規定された本数の矢をすべて無事に的に刺し切ることでステージクリアとなり、次のレベルへと進む。
- 難易度変化:** ステージが進行するにつれて、標的の回転速度の変化、回転方向の反転、初期状態で配置されている障害物（矢）の増加など、複雑な挙動が発生し難易度が上昇する。

## 2. システム機能要件

### 2.1. ゲーム画面・プレイループ機能

- メインゲームキャンバス:** ターゲット（円）の回転アニメーション、および射出された矢の直線移動・固定処理を、低遅延かつ滑らかに描画する。
- タップ/クリック検知:** 画面全体の任意の場所のタップ、またはスペースキーの押下によって、即座に矢を上方へと射出する（入力遅延の極小化）。
- 衝突判定（Collision Detection）:** 射出された矢の先端がターゲットに到達した瞬間、既存の矢の角度データと比較し、一定の許容誤差（ラジアン/度数）以内の場合は「衝突」と判定して即座にゲームオーバー演出へ遷移する。
- 進行管理・スコア:** 現在のステージ数、残りの矢の数、現在までの連続クリアステージ数を画面上に分かりやすくUI表示する。

### 2.2. ユーザー管理・データ同期機能

- 認証システム:** ソーシャルログインまたはメールアドレスを用いた簡単かつ安全なアカウント作成・ログイン機能。

- ・ **ハイスコア・進行度保存:** ユーザーごとの最高到達ステージ数やハイスコアデータをリアルタイムにデータベースへ同期・保存する。
- ・ **リアルタイムランキング（リーダーボード）:** 全ユーザーのハイスコアを集計し、上位プレイヤーのランキングをリアルタイムに表示して競争心を刺激する。

### 3. 技術要件（テクニカル・スタック）

機材環境（Mac環境）および提供された技術構成図に基づき、以下のモダンフルスタック構成を採用する。

| レイヤー       | 選定技術                        | 備考                                |
|------------|-----------------------------|-----------------------------------|
| Framework  | Next.js (App Router)        | 高速表示とServer ActionsによるシームレスなDB操作。 |
| Database   | Supabase (PostgreSQL)       | 認証、DB、リアルタイムランキング機能を統合。           |
| ORM        | Prisma                      | TypeScriptとの親和性が高い型安全な開発。         |
| Deployment | Vercel                      | 東京リージョン（HND1）設定による低遅延配信。          |
| UI/UX      | Tailwind CSS / Lucide React | モダンで清潔感のある、スマホ最適化UI。              |
| App化       | PWA (Progressive Web Apps)  | ホーム画面追加による「アプリ」としての起動、フルスクリーン体験。  |

 **Cursorへのプロンプト指示のポイント:**

Next.jsのApp Router環境下で、フロントエンドのゲーム描画ロジック（HTML5 Canvas または CSS Animation）をクライアントコンポーネント（"use client"）として切り分け、データベース操作やスコア送信を Server Actions で完結させる構造をCursorに意識させてください。

## 4. データモデル (Prisma Schema)

提供されたスキーマ設計の基本原則を踏まえ、本ゲームに必要な「ユーザー認証」「スコア」「ランキング」を管理するためのデータモデルを以下のように定義する。

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

model User {
  id          String   @id @default(cuid())
  email       String   @unique
  name        String   // 初期値はEmailのアカウント名。後で変更可能。
  role        String   @default("user") // "user" or "admin"
  scores      Score[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Score {
  id          String   @id @default(cuid())
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  maxStage    Int       @default(1)        // 到達した最高ステージ
  totalScore  Int       @default(0)        // 累計、または特定モードのハイスコア
  updatedAt   DateTime @updatedAt

  @@index([totalScore(sort: Desc)]) // ランキング高速取得用のインデックス
}
```

## 5. Cursorを用いた開発プロセスへの展開ステップ

この要件定義書ファイルをCursorに読み込ませた後、以下のステップでコード生成を指示することで、スムーズにプロトタイプを構築することが可能である。

1. **ステップ 1 (環境構築・モデル生成):** prisma/schema.prisma に上記のモデルを配置させ、Supabaseへのコネクションおよび初期マイグレーションを実行させる。
2. **ステップ 2 (ゲームコア画面の作成):** src/components/GameCanvas.tsx などのクライアントコンポーネントを作成させ、requestAnimationFrame を用いた目的的回転と、矢の射出口ジック、当たり判定（衝突判定）のコードを一気に生成する。

3. **ステップ 3 (状態管理とServer Actions):** ステージクリア時のスコア更新処理、ゲームオーバー時のデータ送信処理をServer Actions経由でPrismaモデルへと保存するロジックを組ませる。
4. **ステップ 4 (UIのブラッシュアップとPWA化):** Tailwind CSSによるダークモード基調、または鮮やかなカジュアルゲーム風のデザインを適用し、next-pwa 等を使用して manifest.json を構成、スマホのホーム画面にインストール可能な状態にする。